“因光制宜”——融合物理建模与数据驱动方法的光伏发电功率多尺度预测研究

摘要

本文提出了一种融合物理模型与数据驱动的光伏发电功率预测新方法，通过多阶段建模策略系统性地解决了光伏发电功率预测精度不足的关键问题。研究创新性地将太阳位置算法与机器学习优化相结合，并提出了基于场景划分的双层优化框架，为光伏发电功率预测探索了新的技术路径。

针对**问题一**，基于光伏发电物理机理，建立了包含太阳高度角、方位角计算和温度校正的理论模型。通过精确计算阵列面辐照度和组件实际效率，系统分析了理论功率与实际功率的偏差特征。研究发现，模型在温和工作环境下预测误差约为 10%。但在高温条件下，夏季平均相对误差可达 29.95%，这一发现为后续预测模型优化提供了重要依据。研究较为全面地量化了温度非线性效应在不同季节的影响程度，补充了现有研究的不足。

针对**问题二**，构建了 SARIMA(2,1,1)×(1,1,1,96)时间序列预测模型。通过 ADF 平稳性检验和季节性差分处理，模型较好地捕捉了光伏功率的周期特征。在四组不同季节的测试中，模型表现出良好的预测性能，其中 11 月预测的 RMSE 为 0.13，合格率高达 93.80%。这一成果改善了传统时间序列模型在复杂气象条件下的预测效果。

针对**问题三**，提出了基于聚类和机器学习的双层优化策略。研究采用肘部分析法确定聚类数 K=3，依据 NWP 天气特征将数据划分为三类；通过 XGBoost 算法进行残差拟合，使 8 月组预测和合格率提升 5.5 个百分点。这一方法有效改进了异常天气条件下的预测效果，其中 2 月组的合格率提升 8.94 个百分点，展现了算法的适应能力。

针对**问题四**，提出了均值偏差修正的统计降尺度方法，结合随机森林算法优化 NWP 数据。该方法使得辐照度预测的 RMSE 降低 13.80%，在标准测试中的综合评分提升 15.2%。这一进展为改善小尺度区域气象预报精度提供了可行方案，有效弥补了问题一中模型趋势性预测偏差及问题三中模型波动性预测不准的不足。

本研究通过多学科方法的有机结合，构建了一套较为完整的光伏发电功率预测体系。将物理机理与数据驱动相融合，提出了场景划分的双层优化框架，并创新性运用统计降尺度算法优化随机森林预测模型的收敛速度和鲁棒性，提升其泛化能力并避免其特殊情况下陷入局部最优解。研究成果在提升预测精度方面取得了一定成效，为智能电网调度提供了技术支持，具有一定的理论价值和工程应用前景。

关键词：光伏发电功率预测；物理机理建模；SARIMA 时间序列；XGBoost 优化；统计降尺度

1. 问题重述

1.1. 问题背景

光伏发电作为新型清洁能源结构中的重要组成部分，其发电功率受气象条件、地理因素和季节变化等多重因素影响，具有显著的波动性和不确定性。当光伏电站接入电网后，发电功率的波动会对电网的稳定运行带来挑战，因此准确的日前预测可以帮助电网实现功率协调与频率协调，对宏观电力调度至关重要。

然而，云量、阴雨、雾霾等气象因素的随机性使得预测难度增大。为提高预测精度，需要结合历史发电功率数据和数值天气预报（NWP）信息，分析长周期（如季节性变化）和短周期（如日内波动）特性，对发电功率以及诸多气象数据之间的关联展开研究与预测。

1.2. 问题提出

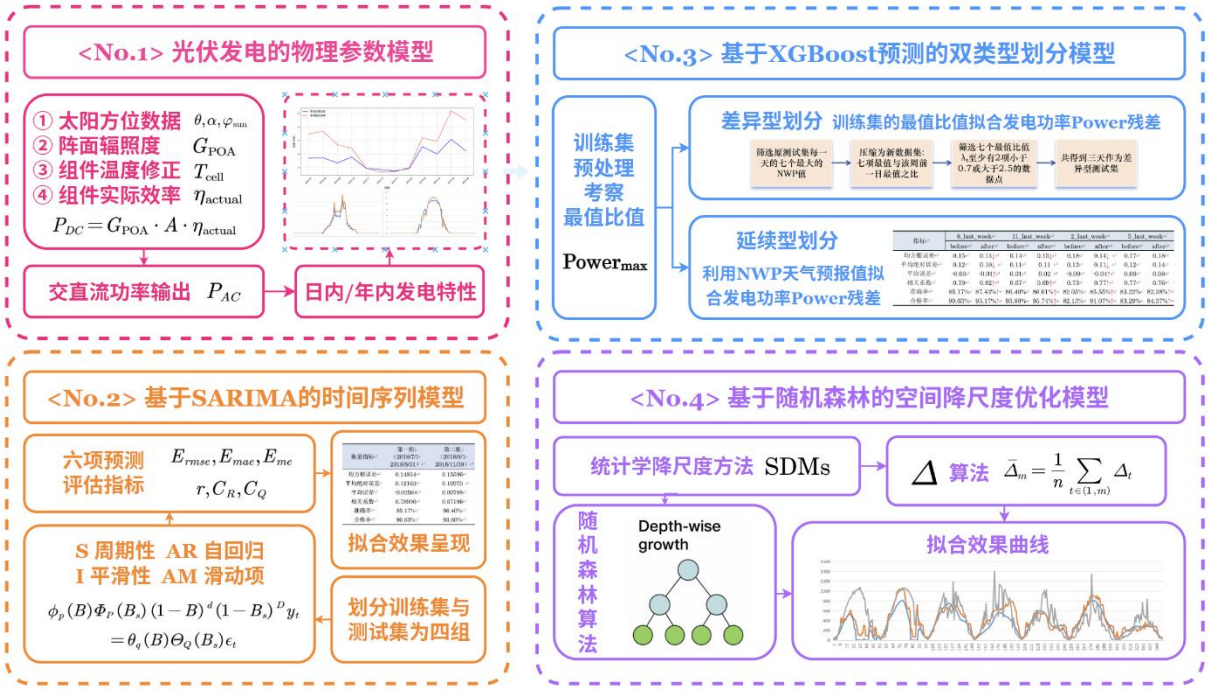
问题 1：本题中需要基于光伏电站的地理位置信息，结合太阳辐照等自然环境数据，计算得到发电机的理论可行功率，不对研究其长周期（季节性）和短周期（日内波动）特性。根据数据集中光电的实际功率与理论可发功率的偏差，评估分析发电功率特性。

问题 2：本题中需要建立基于历史功率的光伏电站日前发电功率预测模型，进行发电功率预测，根据均方根误差 RMSE、平均绝对误差 MAE、平均误差 ME、相关系数 r 以及合格率准确率等因素所谓评判标准，衡量模型的预测效果。

问题 3：本题中需要建立融入了 NWP 数值天气预报信息的光伏电站日前发电功率预测模型，根据预测结果，分析评价 NWP 信息的融入能否有效提高预测精度，并依据数据进行场景划分，寻找拟合效果好的区域场景。

问题 4：由于传统气象预报空间分辨率尺度较大（通常在千米级别），而 MW 级光伏电站覆盖面积可能小于天气预报的空间尺度。因此考虑在现有 NWP 数据的基础上，通过空间插值、统计模型等方法进行空间尺度的细化，以提高光伏功率的预测精度，本文在此需要结合预测结果，检验方法的可行性并分析原因。

图 1：全文整体模型流程图



2. 问题分析

2.1. 问题一：

在本问题中，由题目要求可知，**核心的建模方向即是通过光伏发电的物理学方程与规律，构建物理学模型。**利用数据集中的地理纬度位置（ϕ）、地表环境温度（T_{amp}）、地表总辐照度（G_{GHI}）以及地表散射辐照度（G_{DHI}）等信息，计算得到以 15 分钟为分辨率的理论上的光伏电站发电功率。具体而言，理论模型需考虑太阳位置算法（如太阳高度角、方位角计算）、光伏板倾角和方位角等安装参数，以及温度对光伏组件效率的影响，从而准确模拟光伏电站在理想条件下的发电能力。

随后经过与实测的光伏发电机的发电功率 power 相比较，**分别在长周期（一年 12 月份）与短周期（24 小时）的尺度下比较此理论发电功率与实际发电功率的相对与绝对偏差值，以及其在季度间、日间的发电变化特性。**在长周期尺度下，重点研究季节性变化对发电功率的影响，例如不同季节太阳高度角变化导致的辐照强度差异，以及温度波动对组件效率的长期影响。在短周期尺度下，则聚焦于日内波动特性，分析温度等瞬时变化等因素引起的理论值与实际值的偏差。

2.2. 问题二：

在本问题中，本文需要利用数据集中目标测试集（2、5、8、11 月最后一周）和其余数据构成的训练集，建立基于历史功率的光伏电站日前发电功率预测模型，即利用时间序列模型进行数值的预测。由于本情境下，发电功率随着时间的变化呈现明显的周期性特点，故需要选择一种时间序列模型，同时考量连续性取值以及周期性特征的影响，最终本文选择 SARIMA 模型进行预测工作。此外，在得到测试集的预测结果之后，需要依据要求的六种评估衡量指标进行阐述衡量。**本文中尝试挖掘不同统计学指标在本情境中具有的物理及现实意义，并依据此对拟合效果进行评估分析，结合物理学与气象学规律，讨论本模型的准确性。**

2.3. 问题三：

为了引入 nwp 预测值，提高预测光伏发电站功率的准确性，本文从场景划分入手进行问题分析。首先，本文进行了差异性划分——即通过 NWP 预测值与发电功率在一天之内最大值与前一天内最大值的比值将所有的天数划分为差异性强与差异性弱两类，并通过训练集对差异性强的拟合出比值系数，对差异性强的类别呈上比值系数进行调整。

其次，本文进行了延续性划分——先根据 NWP 预测值进行了聚类分析，并且按照聚类结果，在每一类中分别进行对问题二中的残差进行 XGBoost 机器学习拟合，之后将问题二与问题三的拟合结果求和得到发电功率的预测值，从而提高了模型的精准度与稳定性。

2.4. 问题四：

由于 NWP 预测值的空间尺度过大，无法展现出真实值的细节变化趋势，本文采用统计模型与机器学习方法相结合的方式，对 NWP 预测值进行空间降尺度操作。首先，在统计学降尺度的方法中，本文采用了德尔塔法，该方法速度快，且对数据的依赖程度小，为之后积极学习收敛加速，鲁棒性提升，泛化性提高奠定了良好基础。之后采用随机森林方法对不同站点的 NWP 预测值拟合真实值，从而展现了更为精确的变化关系。通过该模型的预测，丰富了 NWP 预测值的细节性，为进一步提高模型拟合精度提供了可行性。

3. 模型假设与符号说明

3.1. 模型基本假设

(1) 光伏组件的效率与环境温度呈线性关系（温度系数固定为-0.004/℃），以简化计算并符合行业标准，同时避免复杂热力学建模。
(2) 太阳辐照度的理论计算忽略瞬时气象波动（如云层遮挡），仅依赖地理位置和基础气象数据，确保理论功率反映理想发电潜力。
(3) NWP 数据的空间尺度误差可通过统计方法修正（如均值调整或机器学习残差拟合），以适配光伏电站局部气象，同时降低高分辨率数据需求。

3.2. 符号说明

表 1 ：符号说明

公式符号	含义	单位
θ	太阳入射角	弧度
G_{GHI}, G_{DNI}, G_{DHI}	直射、散射、总太阳辐照度	W/m2
G_{POA}	阵列面辐照度	W/m2
T_{cell}	电池组件温度	℃
η_{actual}	组件实际工作效率	
P_{AC}, P_{DC}	组件交流、直流输出功率	MW
B, B_s	延迟算子	

4. 数据预处理

4.1. 数据集选取

在模型建立之前，本文选择查找并使用 PVOD v1.0 数据集，其来源见表 2。

本数据集的有效统计样本量为 n=32928，数据来源光伏电站经纬度坐标为（经度：113.64187；纬度：36.64403），地处**河北省邯郸市涉县索堡镇石家庄村**，与国家公众环境研究中心所提供的公共光伏发电站地图相匹配，见图 2。

图 2 ：Situation07 光伏电站地理位置



表 2 ：

光伏电站的历史发电功率和 NWP 数据规格及要求

数据规格	参数值
数据来源（公开数据集链接）	https://doi.org/10.11922/sciencedb.01094
光伏电站装机容量	20 MW
发电功率和 NWP 数据时间分辨率	采样点/15min
发电功率和 NWP 数据起始-截至时间	2018.07.01- 2019.06.13
NWP 属性	辐射、湿度、温度、气压、风速、风向

此外，本数据集在真实可靠的基础上，还具有以下三点优势，对于本文研究起到很好的辅助作用：

1、本数据集光伏电站的地理位置信息完备，包括准确的经纬度，便于后续对太阳高度角α以及阵列面辐照度G_{POA}的计算，许多相关数据集并不具有详细地经纬度数据，不符合研究需要。

2、本数据集同时兼备预测的数据（“nwp_globalirrad”、“nwp_temperature”等）与实测的数据（“lmd_totalirrad”、“lmd_windspeed”等）两种类型，便于问题 2、3、4 中对本文模型预测结果进行校正，对预测效果进行检验与评估。

3、本数据集包含较齐全的 NWP 数值气象数据，包括辐照度、温度、湿度、风速、风向、气压等等，且时间分辨率为 15 分钟，划分细致便于拟合判断。其他相似数据集多不具备便于后续问题 3、4 中进行模型的细化拟合。

4.2. 数据清洗

考虑到现实情境中自然环境数据的复杂性与异变性，我们需要对下载数据集中的多维度数据进行分析统计，进行缺失值、异常值的检验与处理，以确保对数据的理解使用和模型的构建是有效且准确的。

4.2.1. 缺失值检验

经过比对分析，发现两份数据表格中各列数据均不存在缺失值与零值，其日期对应列均按照时间顺序有序排列，间隔均匀，均为 15 分钟，此外亦无两行完全重复，不需要进行填充与剔除操作。

4.2.2. 异常值检验

本文在此收集整理了不同数据指标的最大值最小值，见表 3。其中按照“预测量”与“实测量”可分为两类，“nwp”表示预测量，“lmd”表示实测量。经过与“国家气象数据中心——河北地面基本气象观测数据”等官方平台参考值作比较，数据集中各类数据均呈现较好的连续性，不存在与主体部分脱节的极大或极小值，例如不存在小于零的发电功率值，不存在小于零的各类辐照强度等。

表 3 ：不同指标的最大值最小值统计

指标(nwp)	globalirrad	directirrad	temperature	humidity	windspeed	winddirection	pressure

最小值	0	0	-11.64	4.85	0.1	0.02	922.34
最大值	964.06	910.09	35.83	100	19.09	359.99	960.28
指标(lmd)	totalirrad	diffuseirrad	temperature	pressure	winddirection	windspeed	power
最小值	0	0	-13.8	925.5	0	0	0
最大值	1518	656	36.099998	966.799988	359	10.1	17.280581

注：globalirrad、totalirrad 表示总太阳辐照度，directirrad 表示直射太阳辐照度，diffuseirrad 表示散射太阳辐照度，temperature 表示环境温度，windspeed 表示地表风速，winddirection 表示地表风向，pressure 表示大气压强，power 表示发电功率。

在进行纵向列比较的同时，本文还对数据集进行横向的准确性考量，即同一行的各类数据值是否符合此时刻河北地区的大体气象条件值，发现本数据集中出现了时间错位的现象，即深夜 0:00 却出现了大于零的光伏发电功率，故选择将 2018.06.30 日的时间戳剔除，并将剩余时间戳依此递补，在最后则顺延 2019.06.13 的时间戳，使得该日的时间序列完成。经过与“河北地面基本气象观测数据”相比对，如此处理有助于提高数据的客观准确性。

5. 模型建立与求解

5.1. 问题一模型建立与分析

5.1.1. 光电站参数呈现

光伏电站发电功率的计算需要基于电站相应的装备参数，整理呈现在此。

表 4 ：光伏发电站参数呈现

参数名称	参数值	参数名称	参数值
组件型号	YL250P-29b	逆变器型号	TC500KH
组件方位	South 31°	温度系数	-0.004/℃
组件个数	80000 台	Pmax	250 Wp
组件面积	1.6335 m^2	T_{NOCT}	46℃
逆变器台数	40 台	逆变器效率	99%

5.1.2. 模型理论分析

光伏发电站的理论功率输出可以通过将特定站点的太阳辐射数据与 PV 组件和逆变器的技术参数相结合来确定。本节将陈述一个系统的物理模型，用于计算理论功率输出，使用光伏站（在此选取 station07，因为其覆盖的时间区域较大）的地理坐标和已知参数作为案例研究。该方法整合了太阳辐照度数据、环境条件和系统特性，以得出发电站总体的交流电（AC）功率输出。

Step 1：获取太阳方位数据

由于本文需要计算太阳辐照度的相关参数，需要获得太阳的方位数据，包括太阳方位角φ_{sun}、太阳高度角α与太阳入射角θ（与组件法线的夹角），公示如下：

$$\sin\alpha = \sin\delta\cdot\sin\phi + \cos\delta\cdot\cos\phi\cdot\cos H$$

其中，ϕ 表示纬度，本题中即为 36.6440°；H 表示时角，反映当前时间与当地太阳正午之间的角度差；δ 表示太阳赤维角，反映太阳与地球赤道之间的角度。后两者的计算公式如下：

$$H = 15° \times (T_{\text{solar}} - 12),\ \delta = -23.44 \times \cos\left(\frac{360}{365}(N+10)\right)$$

其中，T_{solar} 表示当日的小时数，N 表示当日在一年中的天数。由此便可以得到太阳的高度角。利用这些数据，可以进一步求出太阳方位角 φ_{sun} 以及入射角 θ：

$$\cos\varphi_{\text{sun}} = \frac{\sin\delta\sin\phi - \cos\delta\sin\phi\cos H}{\cos\alpha}$$

$$\cos\theta = \sin\alpha\cos\beta + \cos\alpha\sin\beta\cos(\varphi_{\text{sun}} - \varphi_{\text{Panel}})$$

其中，β 表示光伏组件的倾角，在本情境中中取 31°，φ_{Panel} 表示光伏组件的方位角，在本情境中取 180°，即朝向正南方向。

Step 2：利用太阳辐照数据计算阵列面辐照度

在本模型中一共涉及到三种不同类型的太阳辐照度，分别为直接法向辐照度 G_{DNI}（表示垂直于太阳光线的直接辐射）、散射水平辐照度 G_{DHI}（太阳辐射的散射分量），全局水平辐照度 G_{GHI}（在水平表面上接收到的太阳辐射总量）。后两者对应数据集中的 lmd_diffuseirred 和 lmd_totalirrad 两个标签的数据。参考许时佳等[4]在论文使用的方法，三者之间有以下的数学关系式

$$G_{\text{GHI}} = G_{\text{DNI}} \cdot \cos(z) + G_{\text{DHI}}$$

其中，z 表示太阳天顶角，即为太阳高度角 α 的余角。

此外，由于光伏组件通常以固定的倾斜角度安装，以优化太阳能捕获。综合以上数据，可以计算得到阵列面辐照度 G_{POA}：

$$G_{\text{POA}} = G_{\text{DNI}} \cdot \cos(\theta) + G_{\text{GHI}} \times \frac{1+\cos\beta}{2} + G_{\text{GHI}} \times \rho \times \frac{1-\cos\beta}{2}$$

其中，θ 为前文计算得到的太阳入射角，β 为组件倾角 31°，ρ 为地面反射率，用来衡量不同状态的地表对于太阳辐射的散射效果。由图可知，此处光伏电站为植被覆盖的山地，通常可以近似取值 0.2。

Step 3：光伏电池组件温度校正

光伏电池的工作温度会显着影响其效率。电池温度 T_{cell} 可以使用环境温度 T_{amp}、阵列面辐照度 G_{POA} 和组件的标称工作温度 T_{NOCT} 估算，参考国家电网《光伏理论发电功率及受阻电量计算方法》[3]中的公式，呈现如下：

$$T_{\text{cell}} = T_{\text{amb}} + \frac{G_{\text{POA}}}{800}(T_{\text{NOCT}} - 20)$$

其中，T_{amb} 即为数据集中测量得到的环境温度 lmd_temperature。经过查阅 YL250P-29b 光伏组件的性能参数手册，在 NOCT 标称测量条件（太阳辐照数 800W/m^2，环境温度 20℃，风速 1m/s）下，组件的表面温度为 46℃。在此整体公式可以理解为：当辐照为 800 W/m^2 时，电池组件的温度比环境温度高 $T_{\text{NOCT}} - 20$，依据这项差值，进行等比例的加权修正，得到更为准确的 G_{POA} 条件下的电池组件温度。

Step 4：确定组件的实际效率

对于每一块光伏组件，可以用组件效率η_{natual}，标称效率η来衡量其单位面积的发电功率。对于光伏电站的实际运行，两者起到一个**参考值的效果，用于比较不同组件的基础性能，可以用来计算理论可行的发电功率，而不可以代表实际运行的效率**。其表达式如下所示：

$$\eta_{\text{actual}} = \eta \times (1 - \gamma(T_{\text{cell}} - 25))$$

其中，γ表示温度系数-0.004/℃，T_{cell}表示电池组件表面温度，η表示组件的标称效率，其表示组件在 STC 测试条件（太阳辐照值 1000W/m^2，气温为 25℃，）下的效率值，公示如下：

$$\eta = \frac{P_{\max}}{G_{\text{STC}} \times \text{Panel_size}} \times 100\%$$

其中，$P_{\max}$表示组件在 STC 标准测试条件下的峰值功率，经查阅组件参数手册，在此可取 250 Wp；Panel_size 表示单块光伏组件的板面面积，在此取 1.6335 m^2；G_{STC}表示 STC 条件下的太阳辐照度 1000W/m^2。

Step 5：计算交流功率输出

综合以上所有数据，可以根据以下公式[1]计算得到理论的直流输出功率P_{DC}：

$$P_{DC} = G_{\text{POA}} \cdot A \cdot \eta_{\text{actual}}$$

其中，G_{POA}表示组件太阳辐照度；A 表示总的组件面积（组件个数×单个组件面积），取值 1.6335×80000=130680 m^2；η_{natual}表示组件的实际功率。

此外，考虑到光伏点点普遍以交流形式输出电能，故还需要考虑直交流转换以及逆变器容量这一硬性影响。查阅组件手册，在此可以取逆变器效率η_{inverter}为最大效率 99%。同时，经过电站的相关参数计算，可以得到逆变器设备共计 40 台，容配比为 1.0，故最终计算得到最终需要的交流侧总发电功率P_{AC}为：

$$P_{AC} = P_{DC} \times \text{逆变器效率} \times \text{容配比系数}$$

5.1.3. 结果呈现与发电特性分析

本文使用 pylib 库严格复现上述物理模型的公式，每天内计算得到间隔 15 分钟的共 96 个数据值，在此选取数据集中四天，将计算得到的理论数据与实测的发电功率相比较，绘制一组折线图，如图 3：

1、日内数据特征与波动模式

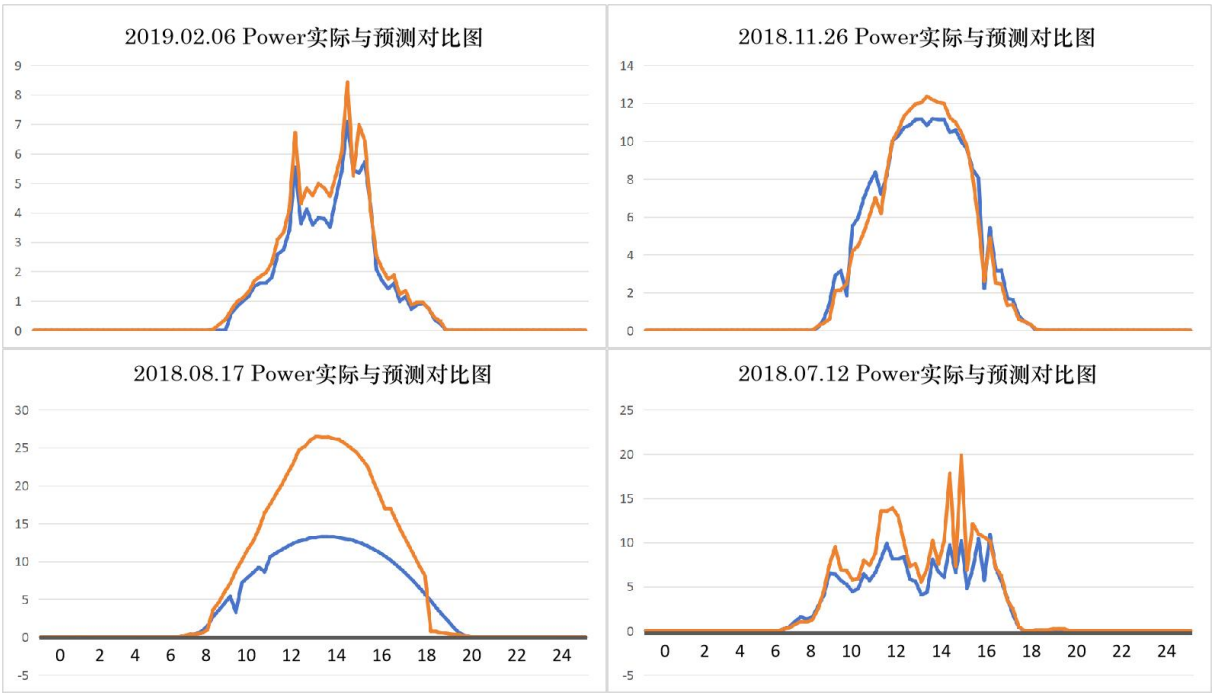
由上图可知，实测功率曲线总体上呈现的“类单峰”结构，峰值出现在太阳辐射强度最大的正午时段（10-14 时），与理论辐照度曲线高度同步。然而，短时气象扰动导致曲线形态显著偏离理想状态：

① **晨昏过渡波动**：日出后 1-2 小时（6-8 时）及日落前（16-18 时），实测功率频繁出现 5%-15%的随机波动，可能与大气散射增强或局部薄云遮挡有关，此时可能会出现不存在直射辐照但存在散射辐照的情况；

② **正午瞬时塌陷**：部分日期（如 2018.11.26）在 12-14 时出现功率断崖式下跌，或在峰值区间内有剧烈的数值波动，10 分钟内降幅超 40%，推测可能由于积云快速覆盖导致辐照度骤降；

③ **午后稳定衰减**：14 时后功率下降速率较快，可能由于太阳辐照度降低，以及组件温度累积升高（典型日升幅达 20-30°C）引发效率衰减有关。

图 3 ：发电功率 Power 实测与预测对比图



故综合来看，模型的预测效果相较实测结果，存在一下几点特征：

① 高温高热的误差特性：在高温（>35℃）或强辐照（>800W/m²）条件下，光伏组件因超温导致效率非线性下降。此时模型采用线性温度修正，无法准确反映实际衰减，造成预测误差显著增大。这主要由于高温下电池载流子复合加剧、能带结构变化等微观机制未被充分考虑，导致预测值普遍高于实测值，见图 3,具体的预测相对差见下表：

表 5 ：各季度发电功率平均偏差

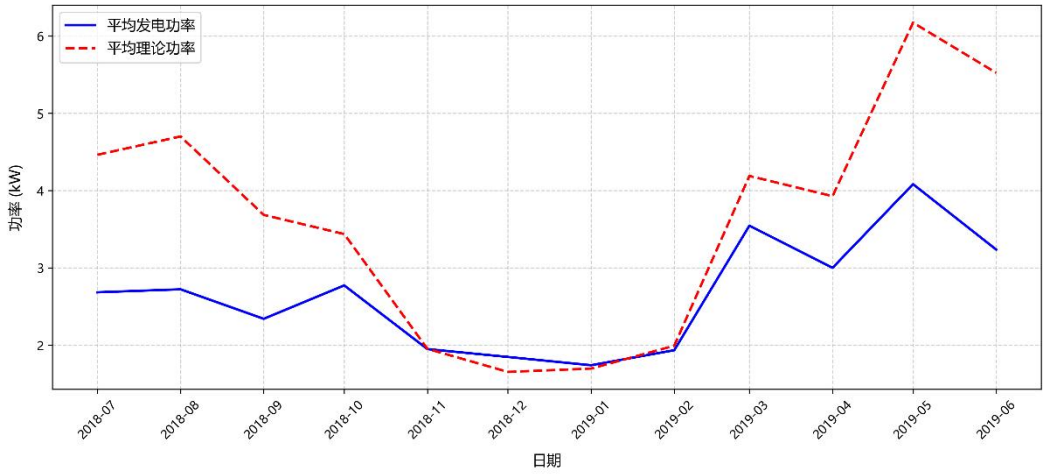
季度	秋季	冬季	春季	夏季
平均相对误差	0.18431	0.12439	0.21355	0.29995

② 稳定工况的准确特性：在辐照 300-600W/m²、温度 15-25℃的温和条件下，光伏系统工作在线性区间，输出功率与辐照呈稳定正比关系。此时电池参数变化平缓，功率波动小于 5%，模型预测误差可控制在 10%内，例如图 3 中右上方的子图。

③ 瞬态过程的匹配特性：当风速>4m/s 引发快速云层移动时，实测功率呈现剧烈波动，1 分钟内可骤降 40-60%。本模型对于这种瞬态响应具有较好的趋势预测效果，但仍然具有前两条特性，功率下降响应快，而恢复过程存在一定程度上的滞后。这源于电池结电容充放电、组件热惯性及局部阴影效应等物理机制的共同作用，导致现有模型难以准确复现实际动态过程。

2、年内数据特征与波动模式

图 4：月平均发电功率与理论功率对比



上图显示，光伏电站月平均发电功率与理论功率均呈现显著的季节性波动特征。2018 年 7 月至 2019 年 6 月间，发电功率在夏季和春末达到峰值，冬季显著下降。2018 年 12 月至 2019 年 1 月，实测功率降至最低，约为 1.6 kW，理论功率亦处于全年最低水平，表明光伏系统发电能力与环境条件密切相关。尽管实测功率整体低于理论值，但二者趋势一致，验证了模型的预测有效性。2019 年 4 月至 5 月，随着光照增强，实测功率快速回升，峰值超过 4 kW，显示出系统具备良好的光照响应能力。

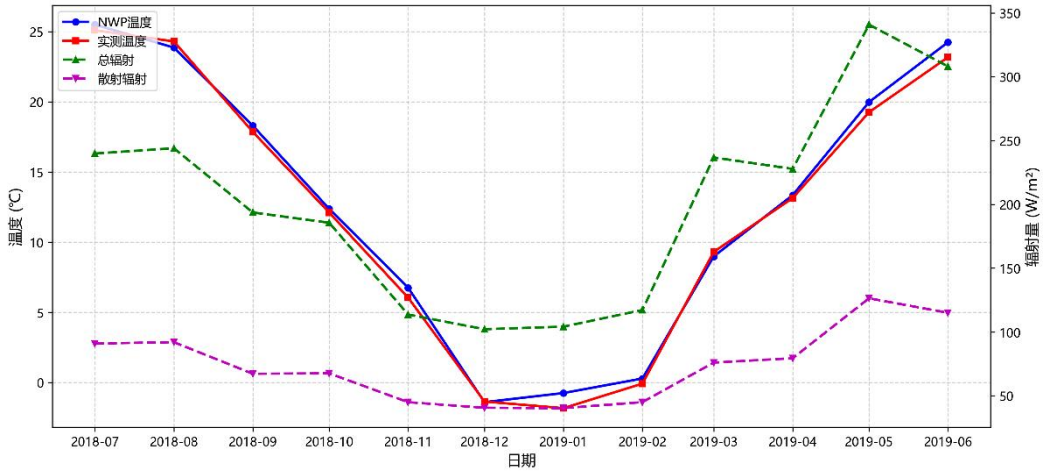
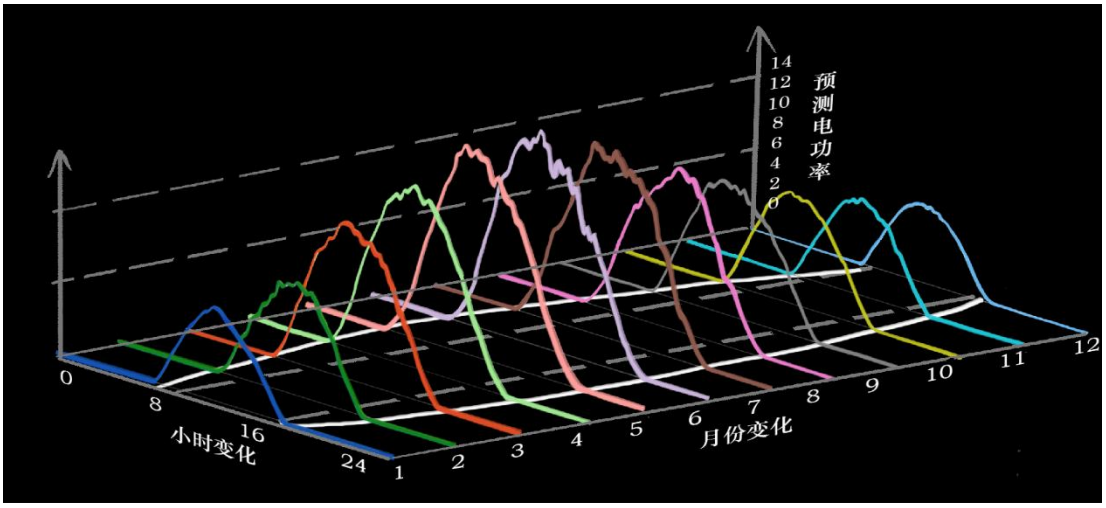


图 5：温度与辐射月平均值变化趋势

上图进一步揭示了发电变化的气象驱动机制。数据显示，总辐射量和倾斜辐射在冬季降至最低（不足 50 W/m²），与发电功率低谷相对应；春季起辐射显著上升，最高接近 350 W/m²，带动发电恢复。温度也呈现季节性波动，冬季低温（接近 0°C）可能影响组件与逆变器性能，略微抑制发电效率。**综上，光伏发电特性主要受太阳辐射驱动，温度为次要影响因素，研究结果对电站运行优化和功率预测具有重要参考价值。**

图 6 ：实际光电功率的三维呈现



5.2. 问题二模型建立与分析

5.2.1. 模型核心思路

在问题二中，本文主要选取了 SARIMA 时间序列模型，利用先前时间步的实测光伏电站发电功率来拟合预测目标日的发电功率。SARIMA 相对其余类型的时间序列预测模型有如下优势。首先，与标准 ARIMA 相比，SARIMA 通过引入季节性差分和季节性自回归移动平均项，能够显式建模光伏数据的多重周期性特征（如日内波动和季节变化），而 ARIMA 仅能处理单一趋势。其次，相对于 Holt-Winters 指数平滑，SARIMA 不仅包含趋势和季节性平滑，还通过自回归和滑动平均项捕捉数据中的动态依赖关系，尤其适合光伏发电中因天气突变导致的复杂波动模式。Holt-Winters 仅依赖三个平滑系数，难以适应辐照快速变化引发的非线性响应。最后，与 LSTM 等深度学习模型相比，SARIMA 在小样本场景下表现更优，且具备参数可解释性。

下面将陈述本模型的构建逻辑：

Step 1：划分数据集与测试集

考虑到时间序列需要将测试集之前的数据作为训练集，才可以对目标的测试集进行预测，故顺应题目要求：将第 2、5、8、11 个月最后一周数据作为测试集，将相邻两组测试集之间的部分作为训练集，共划分为四组进行拟合，在下文中皆以第一组（7 月-8 月）作为案例进行模型的构建分析，该组以 8 月最后一周为测试集，其余三组原理皆相同。

Step 2：构建自回归与移动平均项

本模型是基于 ARIMA 时间序列模型构建的，在此先可以将光伏发电功率视为自由回归（AR）与滑动平均（MA）过程共同驱动的过程。其中自回归部分假设当前功率值 y_t 与与其先前时间步的功率值有关，可表示为：

$$y_t = \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

其中，ϕ_i 为自回归系数，ε_t 为白噪声扰动项。而滑动平均部分则引入过去误差的线性组合。

$$y_t = \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

将两者结合可以得到 ARMA（p,q）模型：

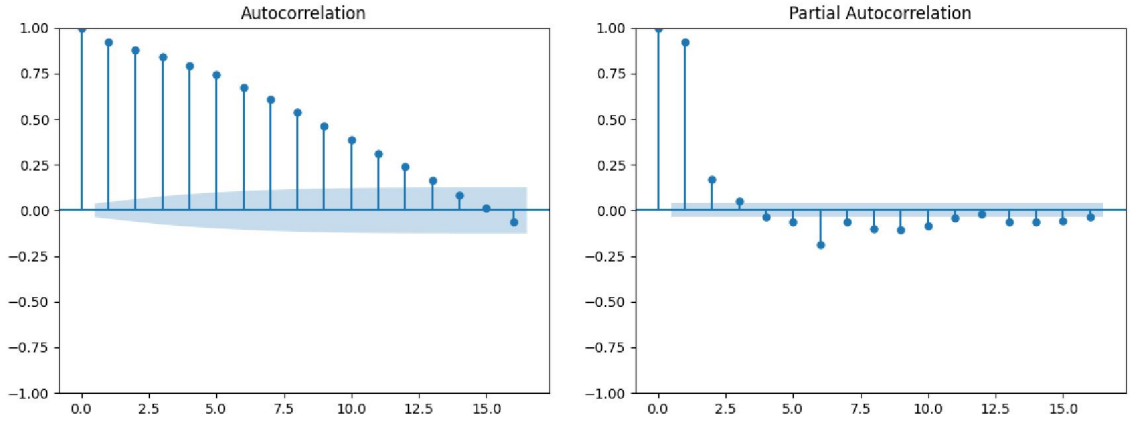
$$y_t = \sum_{i=1}^{p} \phi_i y_{t-i} + \varepsilon_t + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j}$$

其中 p 和 q 为两项滞后阶数分别对应着此时间序列的自相关性与偏相关性，前者表示时间序列与其自身在不同滞后阶数下的线性相关性。在滞后 k 阶时，其定义式为

$$\rho(k) = \frac{\mathrm{Cov}(X_t, X_{t-k})}{\sqrt{\mathrm{Var}(X_t) \cdot \mathrm{Var}(X_{t-k})}}$$

后者篇相关性则用来衡量控制中间滞后项（$X_{t-1}, X_{t-2}, \cdots, X_{t-k+1}$）的影响后，$X_t$ 与 X_{t-k} 的直接相关性，其计算基于前述的 AR 线性回归模型，ϕ_p 即为滞后 p 阶的偏自相关系数。在此以第一组训练集为例，绘制自相关系数与偏相关系数最滞后阶数的变化散点图，如下图所示，其中发现其自相关性随阶数的增大，平滑缓慢下降，呈现拖尾现象而无阶跃，故取 $q=0$ 作为 AR 自回归的滞后阶数；同时偏相关系数在阶数从 1 变化至 2 时快速下降，出现阶跃，故选取阶跃前的横坐标 $p=1$ 作为 MA 滑动平均的滞后阶数。

图 7：自相关性与偏相关性随滞后阶数的变化散点图



Step 3：进行差分 ADF 检验

在本模型中出于对时间序列处理的基本假设，客观物理指标不会再时间很短的范围内有相对极大地突变，所以需要对随时间变化的序列进行平滑性检验。在此采取主流的 ADF 平滑性检验。其核心思想是通过评估数据的频域特性或局部波动性来判断其平滑程度。原假设 H_0 认为信号是平滑的（$R \leqslant \delta$），备择假设 H_1 则认为信号不平滑的（$R > \delta$）。当计算得到的高频能量占比 R 值超过阈值时，就可以在给定的显著性水平下拒绝原假设，判定信号具有显著的非平滑特征。

对于序列检验通过的月份，则直接进行后续操作，对未通过的月份，则进行平滑性处理，$y_t' = y_t - y_{t-1}$，如仍非平稳，可进行多阶差分，差分阶数记为 d，结合步骤 2 的结果，可构建 ARIMA（p,d,q）模型。

Step 4：融入季节性周期数据

光伏发电具有显著的季节性周期性特征，在本模型中主要考虑一天尺度内的周期特征，因此需引入周期性性成分，扩展为 SARIMA 模型，将前若干次周期的发电功率值纳入模型考量。其完整形式为：

$$\mathrm{SARIMA}(p,d,q)(P,D,Q)_s$$

其中$(P,D,Q)_s$为季节性自回归、季节性差分、季节性滑动平均项，s 为季节周期，在本模型中由于预测目标指标仅限于白昼时间段，所以会剔除发电功率为 0 的时间节点：首先会记录一天之内太阳高度角第一次出现由负值变成正值的时间戳刻度为“日出时刻”，太阳高度角第一次出现由正值变成负值的时间戳刻度为“日落时刻”，故选取目标日前七天最小的“日出时刻”和最大的“日落时刻”之间的数据片段作为白昼阶段，由于一周之内日出日落时刻变化波动较小，可以尽可能完整、准确地包含目标日的白昼时间段。由此在每一个数据-训练集组合中，季节周期s为一定值，即为白昼时间段长度。

于是可以顺利构建完整的考虑季节性的时间序列预测模型，在此将 SARIMA 的总表达式呈现如下：

$$\phi_p(B)\Phi_P(B_s)(1-B)^d(1-B_s)^D y_t = \theta_q(B)\Theta_Q(B_s)\epsilon_t$$

其中，B表示延迟算子，表示向过去时间步会不一个时刻；B_s表示周期性延迟算子，把当前序列值的时间向过去回拨了一个周期；$\phi_p(B)$, $\theta_q(B)$表示延迟多项式算子；$\Phi_P(B_s)$, $\Theta_Q(B_s)$表示周期性延迟多项式算子。

5.2.2. 模型效果呈现与分析

沿用上述模型的构建过程，分别将四组训练集进行 SARIMA 时间序列的预测，利用均方根误差、平均绝对误差等指标进行评估衡量，详细数据如下表所示。

表 6 ：SARIMA 时间序列预测效果指标呈现

量指标	第一组（2018/7/1-2018/8/31）	第二组（2018/9/1-2018/11/30）	第三组（2018/12/1-2019/2/28）	第四组（2019/3/1-2019/6/3
方根误差	0.14834	0.13596	0.17949	0.16776
绝对误差	0.12163	0.10970	0.13140	0.12064
均误差	-0.02564	0.00788	-0.09391	0.06077
关系数	0.78906	0.67186	0.73082	0.76765
准确率	85.17%	86.40%	82.05%	83.22%
合格率	90.63%	93.80%	82.13%	83.29%

在此结合本题情景，对这六项评价指标的统计学即物理意义进行分析：

① **均方根误差**E_{rmse}：衡量预测值与实际值之间偏差的“平均幅度”，对较大误差更敏感，在本题中，较低的取值表示模型能较好地避免极端预测偏差，有助于保障调度计划的稳定性；

② **平均绝对误差**E_{mae}：表示预测值与实际值之间的“平均绝对偏差”。它反映了模型在典型时刻的误差水平，更直观地衡量预测值的总体精度；

③ **平均误差**E_{me}：衡量预测值是否存在系统性偏差（偏高或偏低）。在光伏预测中，正值表示模型普遍低估发电功率，负值则表示普遍高估，有助于识别预测的偏向性并进行模型校正；

④ **相关系数r**：描述预测值与实际值在趋势上的同步性，较高的相关系数表明模型能较好地跟随实际功率的变化节奏，尤其重要于动态调度和电网负荷匹配；

⑤ **准确率C_R**：表征模型在实用场景下的可接受度，可直接用于短期功率分配与能源调度。

⑥ **合格率Q_R**：反映模型预测在实际工程应用中的容错能力，适合用于经验判断和辅助决策。

在此呈现这六项衡量指标的理论表达式：

$$\text{均方根误差}E_{rmse}=\sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(\frac{P_{Pi}-P_{Mi}}{C_i}\right)^2};\ \text{平均绝对误差}E_{mae}=\frac{1}{n}\sum_{i=1}^{n}\left|\frac{P_{Pi}-P_{Mi}}{C_i}\right|;$$

$$\text{平均误差}E_{me}=\frac{1}{n}\sum_{i=1}^{n}\left(\frac{P_{Pi}-P_{Mi}}{C_i}\right)$$

$$\text{相关系数}r=\frac{\sum_{i=1}^{n}\left[\left(P_{Mi}-\bar{P}_M\right)\left(P_{Pi}-\bar{P}_P\right)\right]}{\sqrt{\sum_{i=1}^{n}\left(P_{Mi}-\bar{P}_M\right)^2\cdot\sum_{i=1}^{n}\left(P_{Pi}-\bar{P}_P\right)^2}};\ \text{准确率}C_R=(1-E_{rmse})\times100\%$$

$$\text{合格率}Q_R=\frac{1}{n}\sum_{i=1}^{n}B_i\times100\%,\ \text{其中}B_i=\begin{cases}1, & \dfrac{|P_{Pi}-P_{Mi}|}{C_i}<0.25\\ 0, & \dfrac{|P_{Pi}-P_{Mi}|}{C_i}\geqslant0.25\end{cases}$$

故综合以上数值的含义理解，**从表 6 可以看出，SARIMA 模型在不同时间段内对光伏电站发电功率的预测效果总体表现良好，具备较高的拟合精度和稳定性。**就均方根误差（RMSE）与平均绝对误差（MAE）来看，模型在四个阶段中的误差值均维持在较低水平，最低 RMSE 出现在第二组（0.13596），最大出现在第三组（0.17949），与冬季复杂气象条件导致的发电不确定性增加相符。平均误差（ME）指标也呈现出合理的偏差分布，第二组与第四组误差接近零，说明模型在多数时段内未出现系统性偏差。

相关系数方面，四组中均高于 0.67，其中第一组和第四组分别达到 0.78906 和 0.76765，说明预测值与实测值在趋势上高度一致。模型在夏季与春末时段的准确率和合格率最高，分别达 85.17%与 90.63%（第一组）及 86.40%与 93.80%（第二组），验证了在光照资源充足、气象波动较小的时期，模型预测性能更优。相比之下，第三组准确率和合格率相对较低（82.05%、82.13%），也揭示了冬季异常天气对建模精度的挑战。

5.3. 问题三模型建立与分析

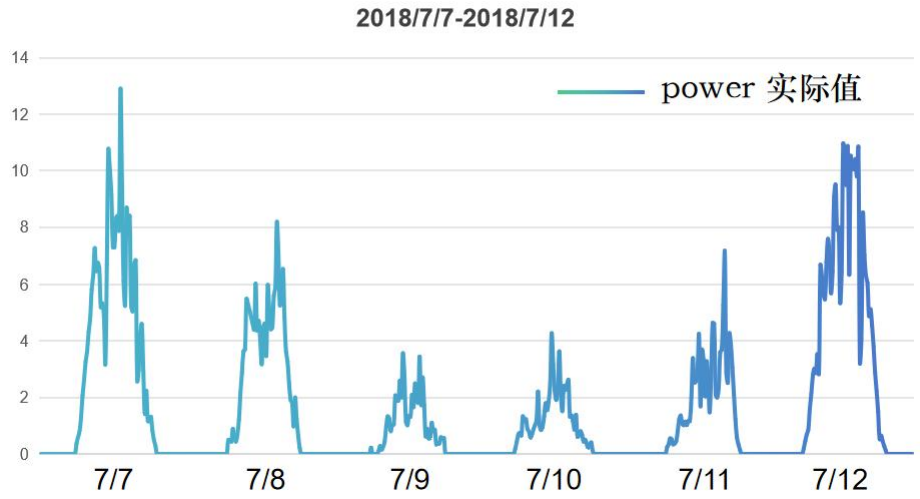
5.3.1. 模型构建

在本问题的情境与要求下，本模型需要将 NWP 数值天气预报数据融入光伏电站的日前发电功率预测模型。希望通过提前预测得到的天气预报值，来辅助发电功率的预测，尽可能提高预测精确度。在此本文构建了**“基于 XGBoost 预测的双类型划分模型”**，构建逻辑呈现如下：

Step 1：条件分析与训练集预处理

首先，本文基于问题二的模型进行了初步的 NWP 融入尝试，发现四组测试集的数据有两组有衡量指标的上升，但增长幅度很小，11 月组为大约 1%，5 月组为大约 2%，况且同时，2 月租和 8 月组却还有大约 3%左右的下降。说明 NWP 的直接拟合并没有很好地反映预测当日天气情况的变化趋势，经过数据集的比对，发现存在一些日期（例如 8 月 25 日），总体太阳辐照度的预测值与实测的总体太阳辐照度差异极大，如下图所示。

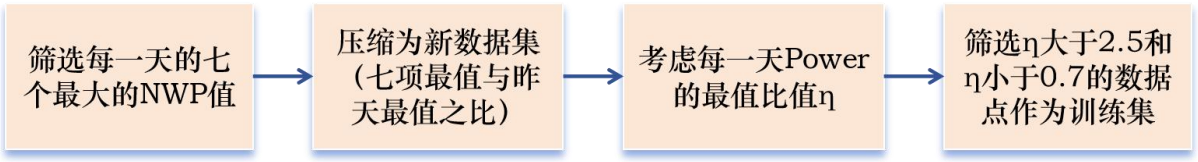
图 8：实际发电功率的周期性差异



此外，由于在 SARIMA 时间序列模型预测下，测试集的数据收到前一天的影响极大，但是有很多相邻周期（此处周期为 24 小时）的差异极大，由于大自然的气候现象多变，不利于整体的拟合，故需要进行分类分情景来处理。

其次，由于情境中模型只有预测值可以作为依据，故本模型试图依据已有的，并且或许存在较大误差的 NWP 天气预报值来进行异常值的划分，在此进行的操作如下：

图 9：训练集预处理流程

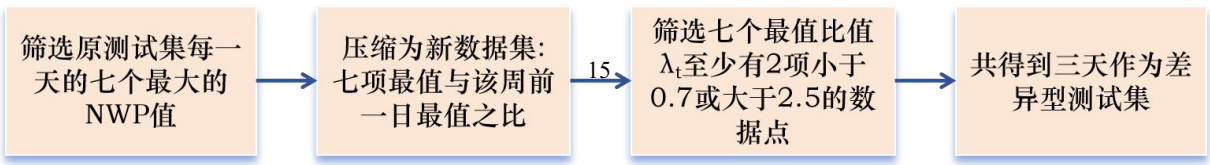


在此，训练集中每一个数据点均由七个最值比值构成，选取每一天的 NWP 最值可以有效突出当日的气候特征，计算每一天的 NWP 最值比值可以有效反映访日气候相对前日的异变程度，如此处理便于后续划分与拟合。

Step 2：测试集差异型划分与 XGBoost 拟合预测

有上述情景可知，选取的四周测试集中存在某些天数的气候异变程度较高，很不利于拟合预测，故在划分训练集时也应对测试集进行针对性划分，划分同样依据 NWP（数值天气预报）数据，具体操作如下图：

图 10　：测试集差异型划分流程



在此，选取七项最值比值中至少有两项小于 0.7 或至少有两项大于 2.5 的数据点，可以在剔除尽可能少的数据基础上，尽可能高效地对测试集进行划分，从而有助于在不同情境下分别对发电功率进行拟合预测，以提高预测精确度。同时，提取出来的三天差异型测试集分别为：8 月 31 日，2 月 26 日和 2 月 27 日，这三天都是最值比值有至少两项小于 0.7，相较模型二的时间序列参照有较大的差异型。此时，采用 XGBoost 集成学习进行残差拟合：**即利用训练集的最值比值拟合发电功率 Power 残差（实测值与模型二的预测值之间的差值）得到一个相对每组测试集标准日（测试周前一天）的比值系数**。随后将这三天的预测值乘以相应的比值系数即可得到模型三的最终预测值，经过与实测值比较后发现这三天差异型数据点所属的“8 月组”和“2 月组”各项预测指标均有显著上升，见表 8。

Step 3：训练集延续型划分与基于聚类的 XGBoost 拟合预测

测试集中剔除差异型数据点后剩余的数据点集合在此命名为“延续型划分”。考虑到不同情景细化的需要，此时本模型依据所有的 NWP 天气预报值，将所有的训练集和测试集进行 K-means 聚类的划分，利用肘部法的思想划分为三类：“寒冷类”、“高温潮湿类”和“高温干燥类”（在此声明高温与寒冷均为相对大小）。

表 7 ：聚类簇中心点参数

簇编号	globalrrad	directrrad	temperature	humidity	windspeed	winddirection	pressure
0	273.83	231.94	**5.14**	32.71	4	185.54	946.26
1	619.53	566.64	**20.59**	**24.87**	5.43	199.95	937.24
2	147.66	114.35	**22.6**	**61.4**	3.88	152.94	933.38

此时模型将基于这三类聚类分别进行 XGBoost 拟合预测，在此则是**利用真实的 NWP 天气预报值拟合发电功率 Power 残差**，得到的校准残差与模型二中的预测值相加即可得到最终模型三的预测值。由于先前在聚类时已经将四组测试集的延续型划分拆分到三组类别之中，故在最后考量结果时，需要将其重拍顺序仍按照四组测试集来进行衡量，由于在差异型划分中考虑了其中两组，在延续型划分中则需要考虑“5 月组”与“11 月组”，其整体预测效果见表 8。

5.3.2. 问题三模型结果分析

在此呈现模型三的两类划分：延续型与差异型的六种拟合评价，将其与模型二中的六项拟合评价指标进行比较，发现几乎绝大多数的误差、标准差都有下降趋势，准确率、相关系数则都有上升趋势。说明本模型在拟合效果上有出色的稳定性和可靠性。

总结来说，本模型主要将数据集进行了两个维度的场景划分，首先会依据天气预报对气候的异常状况（即变化率）进行评估，筛选出突变较多的数据点，其次在剩余的具有较好天气延续型的数据点上，进一步对 NWP 天气数据进行聚类划分，将每一组数据集进行混合预测后，重新依照模型二的条件进行评估，拟合效果极佳。

表 8 ：模型三拟合结果准确率分析

指标	8_last_week	11_last_week	2_last_week	5_last_week

	before	after	before	after	before	after	before	after
均方根误差	0.15	0.13↓	0.14	0.13↓	0.18	0.14↓	0.17	0.18
平均绝对误差	0.12	0.10↓	0.11	0.11	0.13	0.11↓	0.12	0.14
平均误差	-0.03	-0.01↑	0.01	0.02	-0.09	-0.04↑	0.06	0.08
相关系数	0.79	0.82↑	0.67	0.69↑	0.73	0.77↑	0.77	0.76
准确率	85.17%	87.43%↑	86.40%	86.61%↑	82.05%	85.55%↑	83.22%	82.38%↑
合格率	90.63%	95.17%↑	93.80%	95.74%↑	82.13%	91.07%↑	83.29%	84.37%↑

5.3.3. 补充：XGBoost 集成学习方法概述

XGBoost 的核心思想在于：通过不断地不断进行特征分裂来添加树结构，通过不断学习一个新函数来拟合上次预测的残差，使得回归不断优化。由此延伸，对于k棵决策树中样本对应的权重，求和即为所求的预测值。在此将其核心实现步骤进行呈现：

Step 1：数据与模型准备

再进行模型实现之前，需要先对各组数据进行$z-socre$标准化，将数据转换为均值为 0、标准差为 1 的标准正态分布，便于不同数据集间的比较与分析。而后需要划分训练集与测试集，并设定模型的相关超参数，包括树的最大数量、树的最大深度、学习率、正则化项系数等等，用来平衡调控模型的复杂度及拟合精度。

Step 2：样本输入与残差准备

在此，由于本模型需要对整体的的泛函进行优化，故需要先定义目标函数，如下所示，用来衡量树的的综合拟合效果：

$$\mathcal{L}^{(k)}=\sum_{i=1}^{n} l\left(y_i,\hat{y}_i^{(k)}\right)+\Omega\left(f_k\right)$$

该函数前半部分为损失函数项（用来揭示训练误差），后半部分为正则化项（用来惩罚复杂模型）。正则化项值越小复杂度越低，泛化能力越强，表达式为：

$$\Omega(f)=\gamma T+\frac{1}{2}\lambda\sum_{j=1}^{T} w_j^2$$

上式中γ和λ为模型超参数，T为叶子结点的数量，前一项用来控制节点的分裂，w_j为第j个叶子的权重，后一项通过对权重进行 L2 正则化，使得目标函数更倾向于选择权重较小的模型，有利于简化和泛化。

此时输入第一轮样本后，模型会开始进行迭代拟合。在每轮迭代中，根据前一轮的预测结果，计算并记录每个样本的如下两个重要判决量，后续将作用于树的分裂过程：

- 一阶导数 $g_i=\partial\mathcal{L}^{(k)}/\partial\hat{y}_i$，表示梯度，即为房钱预测误差的方向
- 二阶导数 $h_i=\partial^2\mathcal{L}^{(k)}/\partial\hat{y}_i^2$，表示 Hessian 矩阵，即损失函数在此处的曲率

Step 3：分裂候选并寻找最优切分

基于上一步骤生成的决策树，对样本的每一种特征枚举其所有的可能切分点，将每种可能切分之后的的结果划分为“左子节点”和“右子节点”。对每一种切分尝试，计算“是否执行此操作”带来的目标函数增益（Gain）：

$$Gain = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

其中G_L, H_L表示左子集的梯度和 Hessian 二阶导之和，G_R, H_R表示右子集的梯度和 Hessian 之和，λ为 L2 正则化项，γ为最小分裂增益阈值（用来控制剪枝）。

将模型的数据带入上述增益 Gain 计算，在所有特征的左右切分点中找到增益最大的分裂点，若该分裂点的 Gain 小于阈值γ，则不再进行分裂，可以认为构造完成决策树。重复 **Step 2** 和 **Step 3** 过程递归构建左右子树，直到达到最大深度，或增益小于最小阈值，或节点样本数小于设定值。

Step 4：确定叶片权重与模型学习

当树结构固定之后，模型会利用目标函数最小化的原则，为每一片叶子节赋予合适的权重大小，其原理主要是进行二阶的 Taylor 展开操作，通过求导方法得到相应的w_j^*，其简要过程呈现如下：

$$\mathcal{L}^{(k)} = \sum_{i=1}^{n} l\left(y_i, \hat{y}_i^{(k-1)} + f_k(x_i)\right) + \gamma T + \frac{1}{2}\lambda \sum_{j=1}^{T} w_j^2$$

$$\simeq \sum_{j=1}^{n}\left[G_j w_j + \frac{1}{2}(H_j+\lambda) w_j^2\right] + \gamma T$$

其中G_j, H_j的定义同上，在此通过结构函数将投入的数据变量映射到某个特定索引号的叶子节点上去。经过求导可以得到相应的权重：

$$w_j^* = -\frac{G_j}{H_j+\lambda}, \text{ 此时 } \mathcal{L}^{(k)} = -\frac{1}{2}\sum_{j=1}^{T}\frac{G_j^2}{H_j+\lambda} + \gamma T$$

此时前的树结构固定后，对于投入样本的总体预测值将更新为$\hat{y}_i^{(k)} = \hat{y}_i^{(k-1)} + \eta f_k(x_i)$，故最后将森林中$k$棵树所得到的预测值按照学习率权重相加即可得到最终的样本预测值。

5.4. 问题四模型构建与分析

5.4.1. 模型思路构建

在本情境中，文章关注到传统气象预报空间分辨率尺度较大（通常在千米级别），而 MW 级光伏电站覆盖面积可能小于天气预报的空间尺度。这时已有的 NWP 数值天气预报可能会与实际值出现较大的偏差，在此**本模型主要采用统计学降尺度方法以及随机森林机器学习算法，进行降尺度操作，来天气预报的总太阳辐照度进行精确修正**。

首先文本对比分析了几种常见的统计学降尺度方法（Statistical Downscalling Models, SDMs），用以辅助随机森林机器学习算法，经过对比后选择了“Δ方法”，主要原因在于其高效性，可以帮助机器学习损失函数更快地手链，同时增强鲁棒性避免陷入局部最小值。适合于本情境中较为细致的时间间隔划分，有利于多数据的机器学习高效运行。同时“回归+残差”方法使用的模拟真实分布在本情境中存在一定的限制，

即与本模型的十个光伏电站和相应假设不适合；而由于本模型的目标单一，只为了精准预测 07 光伏电站的预报总太阳辐照度，不适合 SAR 算法。

表 9 ：SDMs 统计学方法性能比较

方法	非线性支持	不确定性建模	对数据的依赖	适用情景
Δ 方法	线性修正	较好	低	快速、高效地修正
回归+残差	较好	明确建模	中	模拟真实分布
SAR	较好	可定制	高	多目标优化

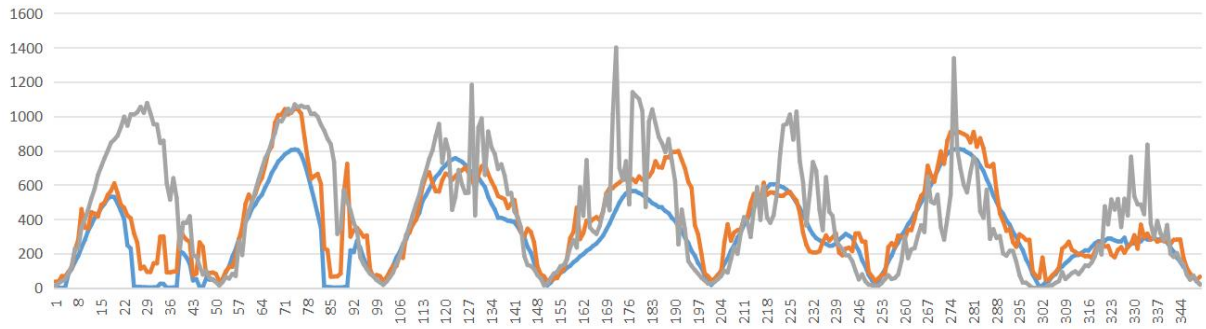
其次，“Δ 方法”的核心运算逻辑如下：对于某个时刻t，计算已有的预测总太阳辐照度与实测总太阳辐照度的插值，记为$\Delta_t = y_{\text{lmd}} - y_{\text{NWP}}$，随后进行平均偏差的处理，在此按照每七项取平均的思路进行处理，公示如下：

$$\bar{\Delta}_m = \frac{1}{n} \sum_{t \in (1,m)} \Delta_t$$

随后，将此均偏差值与t时刻的预测总太阳辐照度相加，得到修正后的预测值。此时，利用随机森林算法，输入值是所有 NWP 数值气象天气预报值与修正后的预测总太阳辐照度，拟合的目标是实测的总太阳辐照度。随机森林的原理和优势在于：随机森林通过对原始数据进行有放回抽样（Bootstrap），训练多棵决策树以构建模型集成。在每棵树的每个分裂节点，只随机选取部分特征来寻找最优分裂，增强各树间的多样性。而最终对分类任务采用多数投票、对回归任务取预测均值，从而降低方差、提高鲁棒性。

5.4.2. 模型结果呈现与分析

图 11　随机森林拟合效果图



在此呈现随机森林的拟合效果折线图，其中纵轴是总体太阳辐照度橙色是空间降尺度之后的随机森林折线图，蓝线是降尺度之前数据集中 NWP 的总体太阳辐照度，灰线是光伏电站实测的总体太阳辐照度。

表 10 使用空间降尺度方法前后G_{DHI}预报值参数变化

指标	空间降尺度前	空间降尺度后	下降率
均方根误差	14.836719	13.042033	13.80%

综上所述并结合模型一模型三，可以得知：图 2 中折线图关系说明模型一可以较好地拟合光伏发电功率的波动性，对于其细微的、高频的变化也有较灵敏的相应，但其缺陷在于对于光滑变化的发电功率，预测值与实测值在相对于绝对误差上均较大，在趋势性上表现不足。此外，结合表 8 中呈现的评估数据可知，模型三的主要作用在于可以更准确地拟合数值，即对于模型一的趋势性欠缺有较好的补充效果，但其对于快速波动信号的波动性拟合有一定的缺陷。故在模型四中，综合了以上两者的优势，如图 10 可知，既可以对细节上快速波动的总体太阳辐照度值有较好的预测，在宏观上对于整体的数值趋势性也有很好的效果，实践了鲁棒性与准确性的统一，具有极高的现实意义，可以一定程度上推动光伏发电站运营的协调与稳定。

6. 模型评价与推广

6.1. 模型的优点

由于初始条件下通过 NWP 得到的估计值与真值之间差异较大，本模型通过两种层次的划分，从而实现了利用 NWP 估计值里面的有效成分，提升了不同情景下光伏发电功率的预测的精度。

此外，本模型通过将统计模型与机器学习相结合，充分利用了前者效率高，速度快，与后者精准度高，拟合度好的优点，从而实现了模型的较快迭代收敛，提高了鲁棒性与泛化能力。

最后，模型具有极好的统一性与整体性，模型四兼具模型一、二、三在拟合趋势性、波动性上的优势，细节上快速波动的总体太阳辐照度值有较好的预测，在宏观上对于整体的数值趋势性也有很好的效果，实践了鲁棒性与准确性的统一，具有极高的现实意义。

6.2. 模型的推广

本研究提出的“因光制宜”光伏功率预测模型具备良好的可扩展性与迁移能力，适用于多区域、多气象条件下的光伏场站功率预测任务。在实际应用中，模型可嵌入智能电网调度系统，实现对新能源出力的高精度预判，提升电网运行的安全性与经济性。同时，结合 NWP 与遥感数据，该框架可拓展至风电预测、负荷预测等领域，构建统一的可再生能源预测平台。未来，可将该模型集成至边缘计算设备中，部署于光伏终端，实现本地化、低延时预测，为新型电力系统提供有力支撑，推动绿色能源的高质量发展。

7. 参考文献

[1] 郑伟烁, 吴芳芳, 郑文悦. 一种光伏电站发电效率多参数综合评估模型及仿真验证[J]. 2021-12-07]. http://kns. cnki. net/kcms/detail/23.1202. TH. 20210727.1345. 004. html, 2021.

[2] Zhou Y, Narsilio G A, Soga K, et al. Achieving Pareto Optimum for Hybrid Geothermal–Solar (PV)–Gas Heating Systems: Minimising Lifecycle Cost and Greenhouse Gas Emissions[J]. Sustainability, 2024, 16(15): 6595.

[3] 国家电网公司. 光伏理论发电功率及受阻电量计算方法（试行）[S]. 北京: 国家电网公司, 2022.

[4] 许时佳. 基于 PEEEMD-BiLSTM-XGboost 光伏发电功率预测方法研究[J]. Advances in Applied Mathematics, 2023, 12: 5039.

附录

附录

基于 pvlib 库的太阳高度角及太阳方位角计算代码

```
import pandas as pd
from pvlib.solarposition import get_solarposition
from datetime import datetime
import matplotlib.pyplot as plt
import matplotlib as mpl

def calculate_solar_elevation():
    # 设置中文字体，防止中文乱码
    mpl.rcParams['font.sans-serif'] = ['SimHei']  # 使用黑体
    mpl.rcParams['axes.unicode_minus'] = False  # 解决负号显示问题

    # 设置位置参数（以北京为例）
    latitude = 36.6440     # 纬度（度）
    longitude = 113.6419  # 经度（度）
    timezone = 'Asia/Shanghai'  # 时区

    # 设置日期时间（可修改为你想要的日期）
    date = datetime(2019, 6, 11, 8, 0)  # 2023 年夏至日中午 12 点

    # 创建时间范围（计算全天变化）
    times = pd.date_range(
        start=date.replace(hour=0, minute=0, second=0),
        end=date.replace(hour=23, minute=59, second=59),
        freq='1H',
        tz=timezone
    )

    # 计算太阳位置
    solar_pos = get_solarposition(times, latitude, longitude)

    # 获取指定时间的太阳高度角
    specific_time = pd.Timestamp(date).tz_localize(timezone)
    elevation = solar_pos.loc[specific_time, 'elevation']
```

```
    print(f"在 {specific_time} 时:")
    print(f"位置: 纬度 {latitude}°, 经度 {longitude}°")
    print(f"太阳高度角: {elevation:.2f}°")
    print(f"太阳方位角: {solar_pos.loc[specific_time, 'azimuth']:.2f}°")

    # 可视化全天太阳高度角变化
    plt.figure(figsize=(10, 6), dpi=100)  # 增加分辨率和尺寸

    # 绘制曲线
    plt.plot(times, solar_pos['elevation'], 'b-', linewidth=2)

    # 设置图表标题和标签
    plt.title(f'太阳高度角变化 - 纬度 {latitude}°, 经度 {longitude}°', fontsize=14)
    plt.xlabel('时间', fontsize=12)
    plt.ylabel('高度角 (度)', fontsize=12)

    # 设置坐标轴
    plt.axhline(0, color='red', linestyle='--', linewidth=1)  # 地平线
    plt.grid(True, linestyle='--', alpha=0.7)

    # 优化 x 轴时间显示
    plt.gcf().autofmt_xdate()  # 自动旋转日期标签

    # 调整布局防止标签被截断
    plt.tight_layout()

    plt.show()

if __name__ == '__main__':
    calculate_solar_elevation()
```

基于 ADF 检验的数据平稳性判断代码+自相关性、偏自相关性检测代码

```
from statsmodels.tsa.stattools import adfuller
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
train = pd.read_csv('train1.csv', parse_dates=['date_time'], index_col='date_time')
result = adfuller(train['power'])
print('p-value:', result[1])  # p-value < 0.05 表示平稳

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_acf(train['power'], lags=16)  # 差分后序列的 ACF
plot_pacf(train['power'], lags=16) # 差分后序列的 PACF
plt.show()
```

基于 SARIMA 的时间序列预测代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error

# 设置随机种子（确保结果可复现）
np.random.seed(42)

# 1. 加载数据
train = pd.read_csv('train1.csv', parse_dates=['date_time'], index_col='date_time')
test = pd.read_csv('test1.csv', parse_dates=['date_time'], index_col='date_time')

# 设置时间序列频率为 15 分钟
train = train.asfreq('15T')
test = test.asfreq('15T')

# 2. 检查数据周期（例如：s=12 表示月度数据的年周期）
print("训练集频率:", train.index.freq)  # 若无频率，需手动设置（如：train = train.asfreq('D')）

# 3. 定义 SARIMA 模型参数
# SARIMA(p,d,q)(P,D,Q)s
order = (6, 0, 0)            # 非季节性部分（AR, I, MA）
seasonal_order = (1, 0, 1, 96)  # 季节性部分（SAR, SI, SMA, 周期 s=96）

# 4. 训练模型
```

```
model = SARIMAX(train['power'],
                order=order,
                seasonal_order=seasonal_order,
                enforce_stationarity=False,
                enforce_invertibility=False)
results = model.fit(disp=False)  # disp=False 关闭迭代日志

# 5. 输出模型摘要
print(results.summary())

# 6. 预测测试集
pred = results.get_forecast(steps=len(test))
pred_mean = pred.predicted_mean  # 点预测
pred_ci = pred.conf_int()        # 置信区间

# 7. 评估指标
mae = mean_absolute_error(test['power'], pred_mean)
rmse = np.sqrt(mean_squared_error(test['power'], pred_mean))
print(f"MAE: {mae:.2f}, RMSE: {rmse:.2f}")

# 8. 可视化
plt.figure(figsize=(12, 6))
plt.plot(train.index, train['power'], label='Train')
plt.plot(test.index, test['power'], label='True Test')
plt.plot(test.index, pred_mean, label='SARIMA Forecast', color='red')
plt.fill_between(test.index,
                 pred_ci.iloc[:, 0],
                 pred_ci.iloc[:, 1],
                 color='pink', alpha=0.3)
plt.title('SARIMA Forecast vs Actual')
plt.legend()
plt.grid()
plt.show()
```

附件 1 评价指标代码实现

```
import pandas as pd
import numpy as np
```

```
# 读取 CSV 文件
data = pd.read_csv("result2.csv")  # 确保文件路径正确

# 提取真实值和预测值
TrueValue = data['TrueValue'].values
Forecast = data['Forecast'].values  # 注意列名是否匹配
n = len(TrueValue)
Ci = 20  # 固定值

# 计算均方根误差(E_max)
numerator = ((TrueValue - Forecast) / Ci) ** 2
E_max = np.sqrt(np.sum(numerator) / n)

# 计算平均绝对误差(E_max_abs)
E_max_abs = np.sum(np.abs((TrueValue - Forecast) / Ci)) / n

# 计算平均误差(E_mean)
E_mean = np.sum((TrueValue - Forecast) / Ci) / n

# 计算相关系数(r)
P_M_mean = np.mean(TrueValue)
P_P_mean = np.mean(Forecast)
covariance = np.sum((TrueValue - P_M_mean) * (Forecast - P_P_mean))
std_dev_M = np.sqrt(np.sum((TrueValue - P_M_mean) ** 2))
std_dev_P = np.sqrt(np.sum((Forecast - P_P_mean) ** 2))
r = covariance / (std_dev_M * std_dev_P)

# 计算准确率(C_R)
C_R = (1 - E_max) * 100

# 计算合格率(Q_R)
B_i = np.where(np.abs(TrueValue - Forecast) / Ci < 0.25, 1, 0)
Q_R = np.sum(B_i) / n * 100

# 输出结果
print("均方根误差(E_max):", E_max)
print("平均绝对误差(E_max_abs):", E_max_abs)
print("平均误差(E_mean):", E_mean)
```

```
print("相关系数(r):", r)
print("准确率(C_R):", C_R, "%")
print("合格率(Q_R):", Q_R, "%")
```

基于 Z-score 的标准化所需训练集均值、标准差计算代码

```
import pandas as pd

# 读取 Excel 文件
file_path = "7_pre_zscore.xlsx"
try:
    data = pd.read_excel(file_path)
except FileNotFoundError:
    print(f"错误：找不到文件 {file_path}")
    exit()

# 排除第一列（假设第一列是日期时间或其他非数值列）
columns_to_analyze = data.columns[1:]

# 计算每列的平均值和标准差
results = {}
for column in columns_to_analyze:
    mean = data[column].mean()
    std = data[column].std()
    results[column] = {"平均值": mean, "标准差": std}

# 输出结果
print("各列统计结果：")
print("{:<25} {:<15} {:<15}".format("列名", "平均值", "标准差"))
print("-" * 55)
for column, stats in results.items():
    print("{:<25} {:<15.4f} {:<15.4f}".format(column, stats["平均值"], stats["标准差
"]))
```

基于 Z-score 的测试集标准化代码

```
import pandas as pd

# 给定的均值和标准差
```

```
stats_dict = {
    'nwp_globalirrad': {'mean': 343.1727, 'std': 241.7124},
    'nwp_directirrad': {'mean': 300.4970, 'std': 232.9818},
    'nwp_temperature': {'mean': 15.4135, 'std': 10.4405},
    'nwp_humidity': {'mean': 39.1603, 'std': 21.9658},
    'nwp_windspeed': {'mean': 4.4104, 'std': 2.4597},
    'nwp_winddirection': {'mean': 179.9403, 'std': 96.3777},
      'nwp_pressure': {'mean': 939.4286, 'std': 7.5670}  # 注意：原数据中是nwp_pressure，这里可能有拼写差异
}

# 读取 Excel 文件
file_path = "8_last_month.xlsx"
try:
    data = pd.read_excel(file_path)
except FileNotFoundError:
    print(f"错误：找不到文件 {file_path}")
    exit()

# 创建一个新的 DataFrame 来存储标准化后的数据
zscore_data = data.copy()

# 对每列进行 Z-score 标准化（第一列除外）
for column in data.columns[1:]:
    if column in stats_dict:
        mean = stats_dict[column]['mean']
        std = stats_dict[column]['std']
        zscore_data[column] = (data[column] - mean) / std
    else:
        print(f"警告：未找到列 {column} 的统计信息，将跳过该列")

# 显示前几行标准化后的数据
print("\n 标准化后的前 5 行数据：")
print(zscore_data.head())

# 可以选择将标准化后的数据保存到新文件
output_path = "8_last_month_zscore.xlsx"
zscore_data.to_excel(output_path, index=False)
print(f"\n 标准化后的数据已保存到 {output_path}")
```

```
# 验证标准化结果（计算新数据的均值和标准差）
print("\n 标准化后各列的统计验证：")
print("{:<25} {:<15} {:<15}".format("列名", "平均值", "标准差"))
print("-" * 55)
for column in zscore_data.columns[1:]:
    if column in stats_dict:
        mean = zscore_data[column].mean()
        std = zscore_data[column].std()
        print("{:<25} {:<15.4f} {:<15.4f}".format(column, mean, std))
```

基于 K-means 的聚类代码

```
#!/usr/bin/env python3
"""
clustering.py: 对 CSV 表格（除第一列）进行聚类操作，并输出聚类结果、肘部法则图以及自动选定的最佳聚类数和簇中心值。

用法示例：
    python clustering.py --input clustering.csv --clusters 4 --output clustered.csv --preclu pre_clu.csv
"""
import argparse
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt

def load_and_prepare_data(filepath):
    df = pd.read_csv(filepath)
    df.set_index(df.columns[0], inplace=True)
    X_raw = df.values
    imputer = SimpleImputer(strategy='mean')
    X_imp = imputer.fit_transform(X_raw)
    return df, X_imp

def main():
    parser = argparse.ArgumentParser(
```

```
        description='对 CSV 文件（除第一列）进行 KMeans 聚类，并生成结果与肘部图')
    parser.add_argument('--input', '-i', default='clustering.csv',
                        help='输入 CSV 文件路径，第一列作为索引')
    parser.add_argument('--clusters', '-k', type=int, default=3,
                        help='指定聚类簇数；若不指定，则使用肘部法自动选择最佳 k')
    parser.add_argument('--output', '-o', default='clustered.csv',
                        help='输出包含聚类标签的 CSV 文件路径')
    parser.add_argument('--max-k', type=int, default=10,
                        help='肘部法则最大聚类数，默认 10')
    parser.add_argument('--preclu', type=str,default='testclu.csv',
                        help='另一个需要分类判断的文件，使用主聚类模型进行预测')
    args = parser.parse_args()

    # 主数据加载与标准化
    df, X_imp = load_and_prepare_data(args.input)
    scaler = StandardScaler()
    X = scaler.fit_transform(X_imp)

    # 计算不同 k 的 WCSS
    wcss = []
    k_range = range(1, args.max_k + 1)
    for k in k_range:
        km = KMeans(n_clusters=k, random_state=42)
        km.fit(X)
        wcss.append(km.inertia_)

    # 绘制肘部图并保存
    plt.figure()
    plt.plot(list(k_range), wcss, marker='o')
    plt.title('Elbow Method for Optimal k')
    plt.xlabel('Number of clusters (k)')
    plt.ylabel('WCSS')
    plt.xticks(list(k_range))
    plt.tight_layout()
    elbow_path = 'elbow_plot.png'
    plt.savefig(elbow_path)
    print(f"肘部图已保存至：{elbow_path}")
```

```
    # 选择最佳 k
    if args.clusters is None:
        diffs = np.diff(wcss)
        optimal_idx = np.argmax(-diffs) + 1
        optimal_k = k_range[optimal_idx]
        print(f"自动选择的最佳簇数: {optimal_k}")
    else:
        optimal_k = args.clusters
        print(f"使用指定的簇数: {optimal_k}")

    # 聚类主数据
    kmeans = KMeans(n_clusters=optimal_k, random_state=42)
    labels = kmeans.fit_predict(X)
    df['cluster'] = labels

    # 打印中心（反标准化）
    centers_scaled = kmeans.cluster_centers_
    centers = scaler.inverse_transform(centers_scaled)
    centers_df = pd.DataFrame(centers, columns=df.columns[:-1])
    print("各簇中心值（未标准化）:")
    print(centers_df)

    # 保存聚类结果
    df.to_csv(args.output)
    print(f"聚类完成，结果已保存至：{args.output}")

    # 如果提供 preclu 文件，则用已有模型预测其类别
    if args.preclu:
        df_pre, X_pre_imp = load_and_prepare_data(args.preclu)
        X_pre_scaled = scaler.transform(X_pre_imp)
        labels_pre = kmeans.predict(X_pre_scaled)
        df_pre['predicted_cluster'] = labels_pre
        df_pre.to_csv('preclu_predicted.csv')
        print("pre_clu.csv 预测完成，类别结果保存为 preclu_predicted.csv")

if __name__ == '__main__':
    main()
```

基于 XGboost 的预测代码

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import xgboost as xgb
import matplotlib.pyplot as plt

# 读取数据
data = pd.read_csv('final_train.csv')
test_data = pd.read_csv('final_test.csv')

# 定义特征和目标变量
X = data.iloc[:, 1:-1]
y = data.iloc[:, -1]

# 数据预处理
# 移除标签中的异常值
mask = (y >= 0) & (y < 1) & (~np.isnan(y)) & (~np.isinf(y))
X = X[mask]
y = y[mask]

# 特征标准化
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# 初始化 XGBoost 回归模型
model = xgb.XGBRegressor(
    objective='reg:squarederror',
    n_estimators=200,
    learning_rate=0.1,
    max_depth=12,
    subsample=0.8,
    colsample_bytree=0.8,
```

```
    random_state=42
)

# 训练模型
model.fit(X_train, y_train)

# 预测
y_pred = model.predict(X_test)

# 评估模型
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"\n 模型评估结果：")
print(f"均方误差 (MSE): {mse:.2f}")
print(f"均方根误差 (RMSE): {rmse:.2f}")
print(f"R² 分数: {r2:.2f}")

# 特征重要性
feature_importance = model.feature_importances_
features = X.columns
importance_df = pd.DataFrame({'Feature': features, 'Importance': feature_importance})
importance_df = importance_df.sort_values(by='Importance', ascending=False)

print("\n 特征重要性：")
print(importance_df)

# 可视化特征重要性
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
plt.show()

# 可视化预测结果
plt.figure(figsize=(10, 6))
```

```
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')
plt.title('Actual vs Predicted Boundary Layer Height')
plt.show()

# 对测试集数据进行预测并输出结果
X_new = test_data[X.columns]  # 确保使用相同的特征列
X_new_scaled = scaler.transform(X_new)
predictions = model.predict(X_new_scaled)

# 保存预测结果
output = test_data.copy()
output['Predicted_BLH'] = predictions  # 添加预测结果列
output.to_csv('final.csv', index=False)
print("\n 预测结果已保存为 'final.csv'")
```

基于随机森林的 NWP 降尺度代码

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. 读取训练集和测试集
df = pd.read_csv('_all.csv', names=['date_time', 'nwp_globalirrad', 'nwp_directirrad', 'nwp_temperature',
                                    'nwp_humidity', 'nwp_windspeed', 'nwp_winddirection',
                                    'nwp_pressure', 'Longitude', 'Latitude', 'target'])
new_data = pd.read_csv('new_data.csv')

# 2. 数据预处理
print("训练集缺失值统计：")
print(df.isnull().sum())

# 删除包含 NaN 的行
df = df.dropna()
print(f"处理后的训练集大小: {df.shape}")
```

```
# 3. 特征和标签
# 确保特征列名一致
feature_columns = ['nwp_globalirrad', 'nwp_directirrad', 'nwp_temperature',
                   'nwp_humidity', 'nwp_windspeed', 'nwp_winddirection',
                   'nwp_pressure', 'Longitude', 'Latitude']

X = df[feature_columns]
y = df['target']    # 使用目标变量列名

# 4. 在训练集内部划分训练和测试集
split_index = int(0.8 * len(X))
X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[:split_index], y[split_index:]

# 5. 创建随机森林模型并训练
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 6. 进行预测（在训练集上评估）
y_pred = model.predict(X_test)

# 7. 模型评估
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"均方误差 (MSE): {mse:.4f}")
print(f"R² 分数: {r2:.4f}")

# 8. 对新数据进行预测
# 确保新数据没有缺失值
new_data = new_data.dropna()

# 使用相同的特征列进行预测
output = new_data.copy()
output['Predicted'] = model.predict(new_data[feature_columns])
output.to_csv('_new_data_predictions.csv', index=False)
print("新数据集预测结果已保存为 _new_data_predictions.csv")
```